

Feature Engineer — Detailed Guide

Retail Demand Forecasting (Walmart M5, Hierarchical) · Team Study Material

1. What this topic is

A machine-learning model cannot look at a calendar and "understand" that Saturdays are busy. It only sees a table of numbers. **Feature engineering** is the work of turning what we know into numbers the model can use.

Feature. One input column the model reads. `lag_7` (what this product sold a week ago) is a feature. **Label / target.** The thing we want to predict — here, units sold on a future day. Features are the question; the label is the answer.

```
RAW TABLE (sales, date, price)
    |
    v
FEATURE BUILDER      compute 38 numbers per row
    |
    v
FEATURE MATRIX  ->  LightGBM
```

2. Why our project needs it

Our raw table says "on 2016-03-05, product FOODS_3_090 in store CA_3 sold 4 units". That single row tells a model almost nothing about **tomorrow**.

What actually predicts tomorrow is context: what this product normally sells, whether it sold at all recently, what day of the week it is, whether it is a SNAP day, what the price is, and which department it belongs to. None of that is in the raw row. We have to build it.

VERIFIED: our champion uses 38 features. *They are listed in CHAMPION_FEATURES in 03_RESEARCH/pipeline/features_v5.py, in a fixed order.*

3. The one rule that governs everything: no leakage

Leakage. *Accidentally giving the model information from *after* the moment it is supposed to be predicting from. A model with leakage looks brilliant in testing and fails completely in reality.*

Every feature we build is computed **strictly at or before the forecast origin** — the last day whose sales we are allowed to know.

There is one deliberate exception, and it is not leakage: **calendar facts about the target day** (which month it is, which day of the month, whether it is a SNAP day, the price that week). Those are genuinely published in advance. A shop knows next Tuesday's date today.

How we proved it. We overwrite every sales value after the origin with 9999, rebuild all the features, and require every one to come back **bit-for-bit identical**. A companion check confirms the *target* does change under the same corruption, so the test cannot pass for a trivial reason.

VERIFIED RESULT: *the corruption test passes for the champion feature builder — all features identical. The foundation suite is 46/46 checks passed.*

4. The seven base groups (32 features)

Our base feature set is organised in groups A–G.

Group	Count	Features
A — Calendar	9	wday, month, year, is_weekend, event_name_1, event_type_1, event_name_2, event_type_2, snap
B — Historical demand	8	lag_1, lag_7, lag_14, lag_28, rolling_mean_7, rolling_mean_28, rolling_std_7, rolling_std_28
C — Recency	3	days_since_last_sale, zero_streak_length, days_since_first_sale
D — Listing	2	days_since_first_listing, pre_listing
E — Price	4	sell_price, recent_avg_price, price_rel_to_recent_avg, price_is_missing
F — Hierarchy	5	item_id, dept_id, cat_id, store_id, state_id
G — Horizon	1	horizon

Plus **6 shape/cycle features** added later (Section 6) = **38 total**.

5. The important features, one by one

Lags — group B

WHAT IS IT? A **lag** is simply "the value N days ago". `lag_7` for today is what this product sold exactly one week ago.

WHY DO WE NEED IT? The strongest predictor of what a product sells tomorrow is what it sold recently. `lag_7` specifically captures weekly rhythm — Saturday looks like last Saturday.

HOW DID WE CREATE IT? By stepping back 1, 7, 14 and 28 days from the forecast origin in the sales matrix. All four are frozen at the origin.

HOW IS IT USED? LightGBM splits on them directly: "if `rolling_mean_28` is below 0.2, predict something near zero."

Rolling statistics — group B

WHAT IS IT? A **rolling window** looks back over the last N days and summarises them. `rolling_mean_7` is the average of the last 7 days; `rolling_std_7` is the **standard deviation** — a measure of how much the values bounce around.

***Standard deviation.** If a product sells 3, 3, 3, 3 the standard deviation is 0 — perfectly steady. If it sells 0, 0, 12, 0 it is large — very spiky.*

WHY DO WE NEED IT? A single lag can be a fluke. An average over 7 or 28 days is the product's actual demand *level*. The standard deviation tells the model how reliable that level is.

HOW IS IT USED? `rolling_mean_28` is effectively the model's estimate of "how big is this product", and the std tells it how much to trust it.

Recency — group C

WHAT IS IT? `days_since_last_sale` counts how long since this product last sold anything. `zero_streak_length` counts the current run of zero days. `days_since_first_sale` is how long the product has existed.

WHY DO WE NEED IT? This is the group that exists **because 68% of our data is zero**. For a product that sells rarely, "how long has it been quiet?" is far more informative than any average.

HOW IS IT USED? It lets the model separate "this product is dead" from "this product is between purchases".

***KNOWN LIMITATION, verified and documented:** `zero_streak_length` largely duplicates `days_since_last_sale`. We measured this and reported it rather than quietly dropping the feature.*

Listing — group D

`days_since_first_listing` and `pre_listing` mark whether the product was even available in that store yet.

WHY? Remember from the data topic: 20.78% of rows have no price because the product had not been introduced. Without this group the model would treat "not stocked yet" as "stocked and sold nothing" — two completely different things.

***KNOWN LIMITATION:** `pre_listing` duplicates `price_is_missing`. Also measured and reported.*

Price — group E

`sell_price` is the price that week. `recent_avg_price` is the average over a trailing **8-week** window (`PRICE_AVG_WINDOW_WEEKS = 8` in `03_RESEARCH/pipeline/config.py`). `price_rel_to_recent_avg` is today's price divided by that average — a promotion signal. `price_is_missing` flags no price.

WHY DO WE NEED IT? Price changes move demand. The **ratio** matters more than the absolute price: £2 means nothing on its own, but "30% below its usual price" means a lot.

IMPORTANT — and this is a question you will be asked. The model uses price as **context**, not as a causal lever. We measured what happens if you simulate a price cut and the response was non-monotone and sometimes economically backwards. So the product deliberately offers **no price what-if slider**. See the Forecasting & Evaluation topic.

Hierarchy — group F

item_id, dept_id, cat_id, store_id, state_id, treated as **categorical** features.

Categorical feature. A label, not a quantity. Store "CA_3" is not "bigger" than store "CA_1". LightGBM handles these natively rather than needing them converted into hundreds of 0/1 columns.

WHY? One global model serves all 30,490 series. Hierarchy is how it knows **which** series it is looking at, and lets it borrow patterns from similar products.

Horizon — group G

WHAT IS IT? horizon is how many days ahead this row is predicting: 1 for tomorrow, 28 for four weeks out.

WHY? Because one model emits all 28 days at once (see the Forecasting topic). Day 28 is harder than day 1, and horizon is how the model knows which it is being asked for.

6. The six shape/cycle features (the "V5" additions)

These take the base 32 to 38. They capture **per-series seasonal shape** — the fact that different products have different weekly and yearly rhythms.

Feature	What it measures
wday_ratio_52w	how this product's sales on this weekday compare to its own average, over ~1 year
wday_ratio_13w	the same over ~13 weeks (more recent, more responsive)
snap_lift	how much this product lifts on SNAP days
weekend_lift	how much it lifts at weekends
month_ratio	how this product behaves in this month of the year vs its own average
dom_ratio	how it behaves on this day of the month (pay-day effects)

HOW WE CREATE THEM. For each series we build a profile over a **728-day (two-year) window ending at the forecast origin** (CYCLE_WINDOW_DAYS = 728). For month_ratio, we take that series' average sales in each month and divide by its overall average:

$$\text{month_ratio}(\text{series}, \text{month}) = \frac{\text{mean sales of that series in that month}}{\text{overall mean sales of that series}}$$

A value of 1.0 means "this month is typical for this product". 1.4 means "this product sells 40% more than usual in this month".

Shrinkage. Low-volume series produce noisy ratios, so profiles are shrunk toward 1.0 based on the series' volume. A product with three sales in two years does not get to claim a strong seasonal pattern.

VERIFIED: the shape features were introduced in Experiment #72 and validated in #73 across 4 windows and 3 seeds; the month/day-of-month pair came from Experiment #74. Measured on the primary window with these features: RMSE 2.1157, MAE 1.0287 for that member.

7. Two feature sets, not one

This surprises people, so be ready for it.

	Direct member	Recursive member
Features	38 (CHAMPION_FEATURES)	32 (REC_COLS_V5)
Where defined	pipeline/features_v5.py	pipeline/champion_blend.py

The recursive member **deliberately excludes six features**:

days_since_last_sale, zero_streak_length, days_since_first_sale, days_since_first_listing, pre_listing, horizon

Why? The recursive model predicts one day, then feeds its own prediction back in to predict the next day. Its prediction is a decimal like 0.3. If 0.3 were fed back as "a sale happened", days_since_last_sale would reset to zero and the recency features would be corrupted a little more with every step of the rollout. horizon is dropped because that model always predicts exactly one step ahead.

This is a genuinely good answer to give a judge. It shows the feature set was designed around how each model actually runs, not copy-pasted.

8. Where this lives in the repository

Path	Role
03_RESEARCH/pipeline/features_v5.py	the champion builder; defines CHAMPION_FEATURES (38)
03_RESEARCH/pipeline/features_v4.py	the shape features wday_ratio_*, snap_lift, weekend_lift
03_RESEARCH/pipeline/features_v2.py	BASE32, the original seven groups
03_RESEARCH/pipeline/champion_blend.py	REC_COLS_V5, the recursive member's 32
03_RESEARCH/pipeline/config.py	PRICE_AVG_WINDOW_WEEKS = 8, HORIZON = 28, seed 42

9. Feature selection — what we did and did not do

We did **not** run an automatic feature-selection algorithm. Features were added in numbered experiments and kept only if they improved measured error across multiple windows and seeds. Experiments #72, #73 and #74 are the ones that produced the six shape features.

We also **kept two features we know are partly redundant** (`zero_streak_length`, `pre_listing`), documented the redundancy, and did not remove them — because the champion was frozen on the measured configuration, and removing a feature would mean revalidating everything.

***Honest limitation:** feature **importance** rankings (which feature mattered most) are not something we have verified from the repository for this guide. Do not quote numbers for it.*

10. Common misunderstandings

"Features are just the raw columns." No. The raw table has sales, date and price. The 38 features are almost all **derived**.

"More features are better." No. The recursive member uses fewer features **on purpose**, and it is the better member on RMSE.

"lag_1 is the most important feature." Not verified. Don't claim it.

"The model understands price causally." No — it uses price as context. Our own measurement showed the price response is non-monotone.

"Calendar features about the future are leakage." No. Dates, events, SNAP days and weekly prices are published in advance. Sales are not.

11. How to present this

"My part is feature engineering — turning raw sales into numbers the model can learn from. We ended up with 38 features in seven groups.

The core ones are lags and rolling statistics: what the product sold 1, 7, 14 and 28 days ago, and its 7- and 28-day average and variability. Then calendar features — weekday, month, events, SNAP days. Then price features, where the useful one isn't the price itself but the price relative to its own 8-week average, which is basically a promotion signal.

Because 68% of our data is zero, we have a whole group about recency — how many days since this product last sold anything. For a product that sells once a month, that matters far more than an average.

We also added six per-series shape features: how *this specific product* behaves on this weekday, in this month, on SNAP days. Those are computed over a two-year window ending at the forecast origin, and shrunk toward neutral for low-volume products so noise doesn't create fake seasonality.

The strict rule is no leakage: every feature is computed at or before the forecast origin. We prove it by overwriting all future sales with 9999, rebuilding the features, and requiring them to come back bit-identical."

12. Likely judge questions

Beginner

Q: What is a feature? A: One input column the model reads. For example `lag_7` — what this product sold a week ago.

Q: How many features do you use? A: 38 for the direct model, 32 for the recursive one.

Q: What's a lag? A: The value N days ago. We use 1, 7, 14 and 28 days.

Technical

Q: How do you know you don't have leakage? A: Every feature is computed at or before the forecast origin. We test it by overwriting all post-origin sales with 9999 and rebuilding — every feature must come back bit-for-bit identical, and we separately confirm the target *does* change so the test can't pass vacuously. The foundation suite is 46/46.

Q: Why do the two models use different feature sets? A: The recursive model feeds its own predictions back in. A fractional prediction like 0.3 would be counted as "a sale happened" and would corrupt `days_since_last_sale` during the rollout, so all recency and listing features are excluded — and `horizon` is dropped because it always predicts one step.

Q: Aren't month and day-of-month leakage, since they're about the future? A: No. Calendar facts are published years in advance. What's forbidden is future *sales*, and none are used.

Q: How did you pick the features? A: Numbered experiments with fixed criteria — a feature stayed only if it improved error across multiple windows and seeds. The six shape features came from Experiments #72–#74.

Q: Any redundant features? A: Yes, two, and we say so: `zero_streak_length` largely duplicates `days_since_last_sale`, and `pre_listing` duplicates `price_is_missing`. We measured it and documented it rather than silently dropping them.

13. Glossary

Term	Meaning
Feature	An input column the model reads
Label / target	What we are predicting (units sold)
Lag	The value N days ago
Rolling window	A summary of the last N days
Standard deviation	How much values bounce around; 0 = perfectly steady
Categorical feature	A label rather than a quantity (e.g. store_id)
Horizon	How many days ahead this row predicts (1–28)
Forecast origin	The last day whose sales we are allowed to know
Leakage	Using information from after the origin — forbidden
Shrinkage	Pulling a noisy estimate toward a neutral value
SNAP	Supplemental Nutrition Assistance Program — a US food benefit flagged in the calendar